第一部分 数据结构与对象

简单动态字符串（SDS）

SDS 的结构定义

- **len**：记录当前字符串的实际长度（不包含 `\0`），获取长度的时间复杂度为 **O(1)**。
- **free**：记录未使用的空间大小，用于优化内存分配。
- **buf[]**：实际存储字符的数组，末尾自动追加 `\0`（兼容 C 字符串函数）。

SDS 的核心特性

- **预分配与惰性释放**
  - **空间预分配**：当 SDS 需要扩容时，Redis 会额外分配未使用空间（`free`）以减少内存重分配次数：

    > 若扩容后 `len` < 1MB：分配 `len` 的等量空间（总空间 = `len` + `free` + 1 = 2 * `len` + 1）。
    >
    > 若扩容后 `len` ≥ 1MB：固定分配 1MB 未使用空间（总空间 = `len` + 1MB + 1）。

  - **惰性空间释放**：当 SDS 缩短时，不立即回收内存，而是通过 `free` 记录剩余空间，供后续操作复用。
- **二进制安全**：SDS 的 `buf` 可以存储任意二进制数据（包括 `\0`），因为长度由 `len` 字段记录，而非依赖 `\0` 判断结束。
- **兼容 C 字符串函数**：SDS 的 `buf` 末尾自动追加 `\0`，可直接使用 `<string.h>` 中的函数（如 `strcmp()`）。
- **杜绝缓冲区溢出**：所有修改操作（如拼接）前会检查剩余空间（`free`），不足时自动扩容。

链表

链表的定义与结构

- **链表结点**
  - **双向链接**：每个节点包含指向前驱和后继的指针，支持双向遍历。
  - **值类型灵活**：`value` 字段为 `void*` 类型，可指向任意数据（字符串、整数、对象等）。
- **链表对象（`list`）**
  - **头尾指针**：直接访问头尾节点，使 `LPUSH`、`RPUSH` 等操作时间复杂度为 **O(1)**。
  - **长度字段**：`len` 直接记录节点数量，无需遍历即可获取长度（时间复杂度 **O(1)**）。
  - **多态性支持**：通过函数指针实现值的复制、释放和比较，支持不同类型数据的存储。

链表的特性

- **双向无环链表**
  - **双向**：每个节点包含前驱和后继指针，支持双向遍历。
  - **无环**：头节点的 `prev` 和尾节点的 `next` 均为 `NULL`，避免循环引用。
- **高效的头尾操作**
  - **头部插入/删除**：直接通过 `list->head` 操作，时间复杂度 O(1)。
  - **尾部插入/删除**：直接通过 `list->tail` 操作，时间复杂度 O(1)。
- **多态性与类型安全**

- **存储任意数据**：节点值通过 `void*` 指针存储，支持字符串、整数、对象等。
- **自定义函数**：通过 `dup`、`free`、`match` 函数实现类型相关的操作，例如：

> `dup`：深拷贝节点值（如复制一个 SDS 字符串）。
>
> `free`：释放节点值占用的内存（如销毁一个 Redis 对象）。
>
> `match`：比较节点值与给定值是否相等。

字典

字典的结构定义

- **哈希表（`dictht`）**
  - **size**：哈希表的大小，初始为 4，每次扩容为当前大小的 2 倍。
  - **sizemask**：哈希掩码，用于计算键的索引（`hash & sizemask`）。
  - **table**：指向哈希桶数组的指针，每个桶是一个链表（链地址法）。
- **哈希节点（`dictEntry`）**
  - **key**：指向键的指针（如 SDS 字符串）。
  - **v**：值的联合体，支持指针、整数、浮点数等类型。
  - **next**：指向冲突链中的下一个节点。
- **字典对象（`dict`）**
  - **ht[2]**：两个哈希表，正常操作使用 `ht[0]`，Rehash 时逐步迁移到 `ht[1]`。
  - **rehashidx**：记录 Rehash 的进度（从 0 到 `ht[0].size-1`）。
  - **type**：指向 `dictType` 结构的指针，包含键和值的操作函数（如哈希函数、键比较函数）。

核心机制与算法

- **哈希函数**：Redis 默认使用 **MurmurHash2** 算法计算键的哈希值。
- **键冲突解决**
  - **链地址法**：哈希冲突的节点通过链表连接，新节点插入链表的**头部**（时间复杂度 O(1)）。
  - **负载因子**：`load_factor = ht[0].used / ht[0].size`，触发扩容的默认阈值是 `load_factor ≥ 1`（可配置）。
- **渐进式 Rehash**：当哈希表需要扩容或缩容时，Redis 通过渐进式 Rehash 避免一次性迁移所有数据导致的阻塞：
  - **准备阶段**：为 `ht[1]` 分配新空间；设置 `rehashidx = 0`，表示 Rehash 开始。
  - **迁移阶段**：Redis 迁移 `ht[0].table[rehashidx]` 的所有节点到 `ht[1]`；`rehashidx` 递增，直到所有桶迁移完成。
  - **完成阶段**：释放 `ht[0]`，将 `ht[1]` 设置为 `ht[0]`；重置 `ht[1]` 为空表，`rehashidx = -1`。

跳跃表Skiplist

跳跃表的定义与结构

- **跳跃表节点（`zskiplistNode`）**
  - **ele**：存储元素的唯一标识（如用户 ID），类型为 SDS 字符串。
  - **score**：排序分值（如用户的积分），支持浮点数。

- **backward**：指向前驱节点，用于从尾到头遍历。
- **level[]**：`forward`——指向后继节点的指针；`span`——当前层到下一个节点的跨度，用于计算元素排名。

- **跳跃表对象（`zskiplist`）**
  - **header**：头节点，包含所有可能的层级（默认 32 层），初始化时各层 `forward` 指向 `NULL`。
  - **tail**：尾节点，支持快速逆序遍历。
  - **length**：记录元素数量，获取长度的时间复杂度为 O(1)。
  - **level**：记录当前跳跃表的最高层，决定查找路径的起始层。

跳跃表的核心特性

- **多层链表结构**
  - **层级随机生成**：每个节点的层数在插入时随机确定（幂次定律：层数越高概率越低），确保高层级稀疏分布。
  - **查找路径优化**：从最高层开始查找，逐步降层，跳过大量节点（类似二分查找）。
- **排序与唯一性**
  - **按分值排序**：节点按 `score` 升序排列，相同 `score` 的节点按 `ele` 的字典序排列。
  - **成员唯一性**：`ele` 唯一，插入重复 `ele` 时直接更新 `score`。
- **范围查询支持**
  - **ZRANGE、ZRANK**：利用 `span` 字段快速计算排名或遍历区间。
  - **ZREVRANGE**：通过 `backward` 指针逆序访问。

Redis 中的应用

- **有序集合（Sorted Set）**
  - **底层实现**：当有序集合元素较多时，使用跳跃表（`zset` 结构包含跳跃表和字典，字典用于 O(1) 查找 `ele` 对应的 `score`）。
  - **典型命令**：`ZADD`——插入元素；`ZRANGE`——按排名范围获取元素；`ZRANK`——获取元素排名。
- **集群节点管理**：集群模式下，跳跃表用于维护槽（Slot）与节点的映射关系。

整数集合

整数集合的结构定义

- **encoding**：编码方式，决定每个元素占用的字节数，`INTSET_ENC_INT16`、`INTSET_ENC_INT32`、`INTSET_ENC_INT64`。
- **length**：集合中元素的数量。
- **contents[]**：元素的实际存储数组，元素按值从小到大排序，且无重复。

核心机制：动态升级

- **触发条件**：当插入一个无法用当前编码表示的整数时，整数集合自动升级编码。
- **升级步骤**
  - **计算新编码**：根据插入值的大小选择最小兼容编码。
  - **扩容内存**：重新分配空间，计算新编码下的总字节数（`length * new_encoding_size`）。
  - **数据迁移**：从后向前依次将旧元素转换为新编码，避免覆盖问题。
  - **插入新元素**：根据新元素的值插入到正确位置（保持有序性）。

- **更新结构**：修改 `encoding` 字段，完成升级。

Redis 中的应用

- **集合键（Set Key）**： 当集合元素全为整数且数量小于 `set-max-intset-entries`（默认512）时，Redis 使用整数集合。
- **性能优化**：小规模整数集合的内存效率远超哈希表（如存储 100个int16仅需 200 + 8 字节，而哈希表需至少 100个entry）。

压缩列表Ziplist

压缩列表的结构

- **头部元数据**
  - **zlbytes**（4字节）： 整个压缩列表占用的内存字节数。
  - **zltail**（4字节）： 尾节点（最后一个Entry）的偏移量（便于快速定位尾节点）。
  - **zllen**（2字节）： 节点数量（若值 ≤ 65535则为实际数量，否则需遍历计算）。
- **节点（Entry）结构**
  - **prevlen**：前驱节点的长度（字节数），用于逆向遍历。
  - **encoding**：当前节点的数据类型和长度编码。
  - **content**：实际存储的数据（整数或字节数组）。
- **结束标记**：0xFF标识压缩列表的结尾。

核心机制与操作

- **编码方式**
  - **整数编码**：根据数值范围选择最小编码（如`int16_t`、`int24_t`、`int32_t`、`int64_t`）。
  - **字符串编码**：短字符串（长度 ≤ 63字节）： `encoding`占1字节（2 + 6）；长字符串： `encoding`占2或5字节，记录长度。
- **连锁更新（Cascade Update）**
  - **触发条件**：插入或删除导致某个节点的`prevlen`从1字节扩展为5字节（或反之）。
  - **影响**：可能引发后续多个节点的`prevlen`更新，最坏时间复杂度为 **O(N^2)**。
  - **优化**：实际场景中概率极低，Redis未做特殊处理。

Redis 中的应用

- **列表键（List Key）**： 当元素均为小整数或短字符串，且数量小于 `list-max-ziplist-entries`（默认512）时，使用压缩列表。
- **哈希键（Hash Key）**： 当字段和值均为小数据，且字段数小于 `hash-max-ziplist-entries`（默认512）时，使用压缩列表。
- **有序集合（Zset）**： 早期版本中，小规模有序集合使用压缩列表存储元素（分值相邻存储），现已被快速列表（Quicklist）替代。

对象

对象类型（`type`）

类型常量	对象类型	底层数据结构	示例命令
`REDIS_STRING`	字符串对象	SDS、整数、embstr 编码的 SDS	`SET`, `GET`, `INCR`
`REDIS_LIST`	列表对象	压缩列表（Ziplist）、双向链表	`LPUSH`, `RPOP`, `LRANGE`
`REDIS_HASH`	哈希对象	压缩列表、字典（Dict）	`HSET`, `HGET`, `HKEYS`
`REDIS_SET`	集合对象	整数集合（Intset）、字典	`SADD`, `SMEMBERS`
`REDIS_ZSET`	有序集合对象	压缩列表、跳跃表（SkipList）+ 字典	`ZADD`, `ZRANGE`

编码方式（`encoding`）

编码常量	说明	适用对象类型
`REDIS_ENCODING_INT`	整数编码（long 类型）	字符串对象
`REDIS_ENCODING_EMBSTR`	embstr 编码的 SDS（短字符串）	字符串对象
`REDIS_ENCODING_RAW`	SDS 字符串（长字符串）	字符串对象
`REDIS_ENCODING_HT`	字典（哈希表）	集合、哈希对象
`REDIS_ENCODING_ZIPLIST`	压缩列表	列表、哈希、有序集合对象
`REDIS_ENCODING_INTSET`	整数集合	集合对象
`REDIS_ENCODING_SKIPLIST`	跳跃表 + 字典	有序集合对象
`REDIS_ENCODING_QUICKLIST`	快速列表（Redis 3.2+，链表+压缩列表）	列表对象
`REDIS_ENCODING_STREAM`	流（Stream，Redis 5.0+）	流对象

对象的核心机制

- **类型检查与命令多态**
  - **类型检查**：执行命令前，Redis 检查操作的键的值对象类型是否匹配。
  - **多态命令**：同一命令对不同编码的对象执行不同操作。
- **内存优化：动态编码转换**

对象类型	初始编码	转换条件	转换后编码
字符串对象	`EMBSTR`	字符串长度增加至 >44字节	`RAW`（SDS）
列表对象	`ZIPLIST`	元素数量 > `list-max-ziplist-entries` 或元素长度 > `list-max-ziplist-value`	`QUICKLIST`
哈希对象	`ZIPLIST`	字段数量 > `hash-max-ziplist-entries` 或字段/值长度 > `hash-max-ziplist-value`	`HT`（字典）
集合对象	`INTSET`	插入非整数元素或元素数量 > `set-max-intset-entries`（默认512）	`HT`（字典）
有序集合对象	`ZIPLIST`	元素数量 > `zset-max-ziplist-entries` 或元素长度 > `zset-max-ziplist-value`	`SKIPLIST` + 字典

- **内存管理：引用计数与共享对象**
  - **引用计数（`refcount`）**

    > 对象被创建时 `refcount=1`。
    >
    > 对象被引用时 `refcount++`（如被加入数据库、被客户端缓存）。
    >
    > 对象被解除引用时 `refcount--`，当 `refcount=0` 时释放内存。

- **共享对象**：Redis 预先创建 `0~9999` 的整数对象供全局共享，减少重复创建。
- **空转时间（`lru`）与淘汰策略**
  - **LRU 模式**：`lru` 字段记录对象最后一次被访问的时间戳（精度为秒），用于 `volatile-lru` 或 `allkeys-lru` 淘汰策略。
  - **LFU 模式**：`lru` 字段分为两部分（高 16 位分钟级时间戳，低 8 位访问频率），用于 `volatile-lfu` 或 `allkeys-lfu`。

第二部分 单机Redis实现

RedisDB

RedisDB的核心结构

- **键空间（dict *dict）**
  - **结构**：哈希表（字典），键为字符串对象（SDS），值为 Redis 对象（字符串、列表、哈希等）。
  - **功能**：存储所有持久化数据，支持增删改查操作。
- **过期字典（dict *expires）**
  - **结构**：哈希表，键与键空间共享（指针引用），值为 `long long` 类型的时间戳（毫秒精度）。
  - **功能**：记录键的过期时间，过期自动删除。
- **阻塞键（blocking_keys）**：用于 BLPOP 等阻塞操作。
- **解锁键（ready_keys）**：解除阻塞的键（与阻塞键配合使用）。
- **监控键（watched_keys）**：被 WATCH 监控的键（用于事务）。

键的过期与删除策略

- **惰性删除（被动删除）**
  - **触发时机**：访问键时检查是否过期（如 `GET`、`HGET` 等命令）。
  - **流程**：执行命令前检查键是否存在于 `expires`；若存在且当前时间 ≥ 过期时间，删除键及其值；返回空值或错误。
  - **优点**：对 CPU 友好，仅在实际访问时处理。
  - **缺点**：若键长期不被访问，可能内存泄漏。
- **定期删除（主动删除）**
  - **触发时机**：Redis 周期性执行 `serverCron` 任务（默认每秒 10 次）。
  - **流程**：每次从 `expires` 中随机抽取 20 个键；若发现过期键，立即删除；当本轮扫描中过期键比例 ≤ 25% 时停止。
  - **优点**：减少内存泄漏风险。
  - **缺点**：需平衡扫描频率与 CPU 开销。
- **内存淘汰策略**
  - **volatile-ttl**：优先淘汰即将过期的键。
  - **allkeys-lru**：淘汰最近最少使用的键。
  - **noeviction**：不淘汰，返回错误（默认策略）。

核心操作流程

- **键的增删改查**
  - **写入操作（如 `SET`）**：将键值对插入键空间（`dict`）；若设置过期时间，同步更新 `expires`。
  - **读取操作（如 `GET`）**：从 `dict` 查找键，若不存在返回 `nil`；若存在，检查 `expires` 是否过期，过期则删除并返回 `nil`。
  - **删除操作（如 `DEL`）**：从 `dict` 和 `expires` 中移除键；释放键对象和值对象的内存（引用计数减 1）。
- **数据库切换（`SELECT` 命令）**：客户端通过 `redisClient.db` 指针指向当前数据库（`redisDb` 数组的索引）。
- **键的阻塞与事务**
  - **阻塞键（`blocking_keys`）**：记录因 `BLPOP` 等命令被阻塞的客户端。
  - **事务监控（`watched_keys`）**：执行 `WATCH` 时记录键，事务提交前检查键是否被修改，实现 CAS（Check-and-Set）。

持久化关联

- **RDB 快照**：保存时遍历 `dict` 中的所有键值对，写入二进制文件。
- **AOF 日志**：记录写命令，重写时根据 `dict` 当前状态生成最小命令集。
- **过期键处理**：
  - **RDB**：持久化时忽略过期键。
  - **AOF**：键过期后追加 `DEL` 命令。

性能优化

- **控制数据库数量**：非必要不使用多个数据库，避免切换开销。
- **合理设置过期时间**：避免大量键同时过期（导致 CPU 突增）。
- **监控内存使用**：通过 `INFO memory` 跟踪 `used_memory` 和 `evicted_keys`。

RDB持久化

RDB 持久化的触发机制

- **手动触发**
  - **SAVE 命令**：阻塞 Redis 主进程直至 RDB 文件创建完成，期间拒绝所有客户端请求（仅调试或内存极小的情况）。
  - **BGSAVE 命令**：主进程 fork 子进程异步生成 RDB 文件，主进程继续处理请求，非阻塞，生产环境首选。
- **自动触发**：通过 `save` 配置项设置触发条件，满足任一条件即触发 `BGSAVE`。
- **主从复制**：主节点首次与从节点同步时触发 RDB 生成。
- **SHUTDOWN 命令**：正常关闭服务器时自动执行 `SAVE`（若未开启 AOF）。

RDB 文件生成流程

- **子进程创建**
  - **写时复制（Copy-on-Write）**：`BGSAVE` 通过 `fork()` 创建子进程，子进程共享主进程内存页。
  - 当主进程修改数据时，内核复制被修改的页，确保子进程的数据视图冻结在 fork 瞬间。
- **数据序列化**：子进程遍历当前数据库的所有键值对，按 RDB 格式序列化数据。
  - **键空间遍历**：按字典顺序扫描数据库的 `dict`。
  - **数据编码**：根据数据类型选择最优编码（如字符串直接存储，列表转为压缩列表等）。
- **文件写入**
  - **临时文件**：数据先写入临时文件（默认 `temp-<pid>.rdb`），完成后原子替换旧 RDB 文件。
  - **刷盘策略**：依赖操作系统缓冲区，可通过 `rdb-save-incremental-fsync` 配置增量刷盘。
- **错误处理**
  - **子进程崩溃**：主进程记录日志，RDB 文件保留旧版本。
  - **磁盘满**：写入失败，删除临时文件，保留旧 RDB。

RDB 文件结构

- **魔数**：5字节，固定为 `REDIS`，标识文件类型。
- **RDB 版本号**：4字节，如 `0009` 表示 Redis 6.x 的 RDB 版本。
- **数据库数据**：按数据库编号顺序存储各库的键值对。
- **键值对编码**：类型标识（1字节）+ 键长度 + 键内容 + 值编码（如字符串长度 + 内容）。
- **过期时间**：可选字段，记录键的毫秒级过期时间戳。
- **结束标识**：1字节 `0xFF`。
- **CRC64 校验和**：8字节，用于验证文件完整性。

RDB 文件加载流程

- **文件校验**：检查魔数和 CRC64 校验和。
- **按序解析**：逐个读取数据库数据，重建键空间和过期字典。
- **阻塞加载**：加载期间拒绝所有客户端请求，直至完成。

RDB 与 AOF 的对比

特性	RDB	AOF
持久化粒度	时间点快照	记录每次写操作（日志追加）
数据安全性	可能丢失数分钟数据	通常最多丢失1秒数据（默认配置）
恢复速度	快	慢（需重放日志）
文件体积	小	大（需定期重写优化）
适用场景	备份、快速重启、容忍数据丢失	高数据安全性要求

AOF持久化

AOF 持久化的工作机制

- **命令追加（Append）**
  - 客户端执行写命令（如 `SET`、`HSET`）后，Redis 将该命令转换为 Redis 协议格式（RESP）。
  - 命令追加到 `aof_buf` 缓冲区（内存）。
  - 根据配置的同步策略（`appendfsync`），将 `aof_buf` 写入并同步到 AOF 文件。
- **文件同步策略（`appendfsync`）**

策略	行为	数据安全性	性能
`always`	每个写命令都同步到磁盘（调用 `fsync`）	最高（零丢失）	最差（频繁 IO）
`everysec`	每秒批量同步一次（后台线程执行）	最多丢失1秒数据	平衡（默认配置）
`no`	由操作系统决定同步时机（通常约30秒）	最低	最佳

AOF 文件重写（Rewrite）

- **触发条件**
  - **手动触发**：执行 `BGREWRITEAOF` 命令。
  - **自动触发**：根据配置的阈值自动触发。
- **重写流程**
  - **创建子进程**：主进程 fork 子进程执行重写（利用写时复制保证数据一致性）。
  - **遍历数据库**：子进程遍历所有数据库的键空间，生成重建数据的最小命令集。
  - **优化策略**：合并多个命令（如用 `HMSET` 替代多次 `HSET`）；忽略过期键和已删除键的历史操作。
  - **写入临时文件**：新 AOF 命令写入临时文件（避免影响原文件）。
  - **替换旧文件**：重写完成后，用临时文件原子替换旧 AOF 文件。

- **重写期间的写入处理**
  - **双缓冲区机制**：主进程将重写期间的写命令同时写入 `aof_buf`（原AOF文件）和 `aof_rewrite_buf`（重写缓冲区）。
  - **同步阶段**：重写完成后，主进程将 `aof_rewrite_buf` 追加到新 AOF 文件，确保数据一致性。

AOF 文件加载与数据恢复

- **创建伪客户端**：模拟客户端执行 AOF 文件中的命令。
- **逐条执行命令**：按顺序重放所有写操作，重建内存数据库。
- **错误处理**：若 AOF 文件损坏（如末尾不完整），可用 `redis-check-aof --fix` 工具修复；Redis 4.0+ 支持加载截断的 AOF 文件。

混合持久化（RDB + AOF）

- **文件结构**：重写后的 AOF 文件前半段为 RDB 格式快照，后半段为增量 AOF 命令。
- **优点**：快速加载（RDB 部分）+ 低数据丢失风险（AOF 部分）；文件体积小于纯 AOF。
- **启用方式**：设置 `aof-use-rdb-preamble yes`。

生产环境建议

- **启用 AOF + 混合持久化**：兼顾安全性与恢复速度。
- **监控 AOF 状态**：通过 `INFO persistence` 查看 `aof_current_size` 和 `aof_rewrite_in_progress`。
- **合理配置同步策略**：对数据安全性要求极高：`appendfsync always`；平衡场景：`appendfsync everysec`。
- **定期备份 AOF 文件**：与 RDB 文件一同存储至异地。

事件处理

- Redis 采用 **单线程事件驱动模型**，通过高效的事件循环处理网络 I/O 和定时任务，实现高并发与低延迟。

文件事件（File Events）

- **功能**：处理客户端请求、命令读取与响应等网络 I/O 操作。
- **实现**：基于 **I/O 多路复用**（如 `epoll`、`kqueue`）监听多个套接字，实现非阻塞通信。
  - **连接处理**：客户端发起连接时触发 `accept` 事件，创建连接对象。
  - **命令处理**：客户端发送命令后触发 `read` 事件，解析并执行命令，结果写入输出缓冲区，触发 `write` 事件返回响应。
- **特点**：单线程按序处理，避免竞态条件，依赖高效的数据结构与协议解析。

时间事件（Time Events）

- **功能**：执行定时或周期性任务（如清理过期键、持久化、统计等）。
- **类型**：
  - **周期性事件**：如 `serverCron`（默认每秒 10 次），负责键过期检查、AOF 重写触发、集群心跳等。
  - **一次性事件**：如延迟执行的任务。
- **调度**：事件按执行时间戳排序，每次事件循环选择最近的事件执行。

事件循环（Event Loop）

- **等待事件**：调用 I/O 多路复用 API（如 `epoll_wait`），阻塞至文件事件就绪或最近时间事件到达。
- **处理文件事件**：执行就绪的读写事件（如接受连接、读取命令、返回响应）。
- **处理时间事件**：执行所有到期的时间事件（如 `serverCron`）。
- **循环重复**：进入下一轮事件等待。
- **优先级**：优先处理文件事件（保证响应速度），时间事件可能被延迟但不会丢弃。

设计优势

- **高效单线程**：避免多线程上下文切换与锁竞争，内存操作快速。
- **无阻塞**：I/O 多路复用 + 非阻塞套接字，充分利用 CPU。
- **可扩展性**：通过合理拆分任务（如后台持久化使用子进程），避免主线程阻塞。

Redis客户端

客户端生命周期与核心流程

- **连接建立**
  - 客户端通过 TCP 连接服务器，触发文件事件 `accept`。
  - 创建 `client` 对象，初始化 `fd`、`db`（默认选库 0）、`querybuf` 等字段。
  - 将客户端加入服务器的全局客户端列表 `clients`。
- **命令处理**
  - **读取请求**：客户端发送的命令数据被读入 `querybuf`，按 Redis 协议（RESP）解析为 `argv` 和 `argc`。
  - **查找命令**：根据 `argv[0]`（命令名）在命令表 `commandTable` 中查找对应的 `redisCommand` 结构（含函数指针）。
  - **执行命令**：调用 `cmd->proc(client)` 执行命令，结果写入 `reply` 缓冲区。
  - **返回响应**：将 `reply` 中的数据通过套接字发送给客户端，清空缓冲区。
- **连接关闭**
  - **主动关闭**：客户端发送 `QUIT` 命令或断开连接。
  - **被动关闭**：服务器因超时（`timeout` 配置）、协议错误或内存限制强制断开。
  - **资源释放**：释放 `client` 对象、缓冲区内存，关闭套接字。

客户端状态与特性

- **阻塞操作**
  - **场景**：执行 `BLPOP`、`BRPOP`、`SUBSCRIBE` 等阻塞命令。
  - **处理**：将客户端标记 `CLIENT_BLOCKED`，移入 `blocked_clients` 列表；阻塞期间暂停读取新命令，直到数据到达或超时。
- **事务支持（MULTI模式）**：命令缓存在客户端的 `mstate` 队列，直到 `EXEC` 或 `DISCARD`；通过 `WATCH` 监控键，被修改时事务中止。
- **Pub/Sub 订阅**
  - **订阅状态**：客户端订阅频道后，只能执行订阅相关命令，其他命令被拒绝。
  - **数据推送**：发布的消息直接写入客户端的 `reply` 缓冲区。

- **输出缓冲区管理（限制配置）**：超出限制时，服务器断开连接以防止内存耗尽。

客户端的限制与优化

- **最大连接数**：`maxclients 10000`（默认 10000），超出限制时，新连接被拒绝。
- **超时控制**：`timeout 300`（默认 300 秒无交互后断开连接）。
- **客户端列表查询**：`CLIENT LIST` 查看所有客户端信息（IP、状态、内存使用等）。

Redis服务器

服务器启动流程

- **初始化配置**：加载 `redis.conf` 配置（端口、持久化策略、内存限制等）；创建默认的 **数据库**，初始化键空间字典与过期字典。
- **启动事件循环**：创建 **事件循环器**（`aeEventLoop`），绑定文件事件（网络 I/O）和时间事件；监听客户端连接请求。
- **加载持久化数据**：若存在 `dump.rdb` 或 `appendonly.aof`，恢复数据到内存。
- **运行就绪**：进入事件循环（`aeMain`），开始处理客户端请求与后台任务。

请求处理流程

- **接收连接**：客户端发起连接，触发文件事件，调用 `accept` 创建 `client` 对象，加入全局客户端列表。
- **读取命令**：将客户端发送的数据存入 `querybuf`，按 RESP 协议解析为命令参数（`argv` 和 `argc`）。
- **执行命令**：查找命令表获取对应的处理函数；执行函数，操作数据库（如更新键空间），结果写入 `reply` 输出缓冲区。
- **返回响应**：将 `reply` 中的数据通过套接字返回客户端，清空缓冲区。

多数据库管理

- **数据库切换**：通过 `SELECT` 命令切换当前数据库（修改 `client.db` 指针）。
- **键空间操作**：所有数据操作基于 `redisDb.dict`（键空间字典）和 `redisDb.expires`（过期字典）。
- **键过期策略**：结合惰性删除（访问时检查）和定期删除（`serverCron` 任务扫描）。

后台任务与扩展功能

- **持久化**：**RDB**—通过 `BGSAVE` fork 子进程生成快照；**AOF**—追加写命令，定期重写优化文件。
- **过期键清理**：`serverCron` 每秒执行 10 次，随机抽查并删除过期键。
- **内存管理**：根据 `maxmemory` 策略（LRU、LFU 等）淘汰键，防止内存溢出。
- **慢查询日志**：记录执行超时的命令（`slowlog-log-slower-than` 配置）。

单线程模型的核心优势

- **无锁设计**：避免多线程竞争，简化实现，内存操作原子化。
- **高效 I/O**：通过 **I/O 多路复用**（如 `epoll`）处理高并发连接。
- **CPU 亲和性**：单线程绑定 CPU 缓存，减少上下文切换开销。

第三部分 Redis分布式

Redis Sentinel（哨兵）

Sentinel 的工作机制

- **故障检测（Failure Detection）**
  - **主观下线（Subjective Down）**：单个 Sentinel 实例检测到主节点在30 秒内无响应，标记其为 `SDOWN`。
  - **客观下线（Objective Down）**：当超过 `quorum` 的 Sentinel 实例确认主节点不可达，标记为 `ODOWN`，触发故障转移。
- **领导者选举（Leader Election）**
  - **Raft 算法**：Sentinel 使用类似 Raft 的算法选举领头 Sentinel（Leader），由 Leader 执行故障转移。
  - **选举条件**：获得多数 Sentinel 实例的投票（`> N/2 + 1`，N 为 Sentinel 总数）；避免多个 Sentinel 同时发起故障转移。
- **故障转移（Failover）流程**
  - **选择新主节点**：Leader Sentinel 根据规则从从节点中选出新主节点：

    > 优先级（`slave-priority` 配置）；复制偏移量最大（数据最新）；运行 ID 字典序最小（最终裁决条件）。

  - **提升新主节点**：向目标从节点发送 `SLAVEOF NO ONE`，使其成为主节点；等待新主节点确认角色切换。
  - **切换从节点复制关系**：向其他从节点发送 `SLAVEOF` 命令，使其复制新主节点。
  - **更新配置**：Sentinel 更新监控的主节点地址，并通知客户端。
- **客户端服务发现**
  - **连接流程**：客户端向 Sentinel 查询当前主节点地址；客户端直接连接主节点，失败时重试查询。
  - **SDK 支持**：主流 Redis 客户端库（如 Jedis、Lettuce）内置 Sentinel 集成。

Sentinel 的架构与部署

- **Sentinel 部署模式**
  - **推荐配置**：至少部署 **3 个 Sentinel 实例**（奇数个，避免脑裂），分布在独立物理节点。
  - **网络拓扑**：Sentinel 实例与 Redis 节点部署在同一网络，避免分区误判；Sentinel 之间通过 **Gossip 协议**通信，共享节点状态。
- **Sentinel 与 Redis 节点的关系**
  - **监控对象**：每个 Sentinel 监控 **一个主节点及其所有从节点**。
  - **自动发现**：Sentinel 通过主节点获取从节点列表，并持续监控其状态。

Sentinel 的优缺点

- **优点**
  - **自动化容灾**：无需人工干预即可完成故障转移。
  - **高可用**：Sentinel 自身多实例部署，避免单点故障。
  - **客户端透明**：客户端通过 Sentinel 自动发现主节点，业务代码无需硬编码地址。
- **缺点**
  - **数据一致性**：异步复制可能导致故障转移后数据丢失（已提交但未同步到从节点的数据）。
  - **复杂度**：需部署多个 Sentinel 实例，配置和维护成本较高。

- **脑裂风险**：网络分区可能导致多个主节点并存（需合理配置 `quorum` 和节点分布）。

生产环境建议

- **部署最佳实践**
  - **Sentinel 数量**：至少 3 个实例，部署在独立物理节点或可用区。
  - **网络优化**：确保 Sentinel 与 Redis 节点间低延迟通信，避免跨地域部署。
  - **监控告警**：监控 Sentinel 日志和 `INFO Sentinel` 输出，关注 `odown` 事件和故障转移次数。
- **避免脑裂的配置**
  - **合理设置quorum**：通常设为 `N/2 + 1`（N 为 Sentinel 总数）。
  - **调整min-slaves-to-write**：主节点需至少同步到指定数量的从节点才接受写操作。
- **客户端容错**
  - **重试策略**：客户端应实现重试逻辑，处理故障转移期间的短暂不可用。
  - **多语言 SDK**：选择支持 Sentinel 的客户端库（如 Java 的 Jedis、Python 的 redis-py）。

常用命令

- **SENTINEL masters**：查看主节点信息。
- **SENTINEL slaves {master}**：查看从节点信息。
- **SENTINEL failover {master}**：强制触发故障转移（无需ODOWN）。
- **SENTINEL sentinels {master}**：查看 Sentinel 节点列表。

Redis集群

集群的架构与数据分片

- **哈希槽（Hash Slot）分配**
  - **分片规则**：对键的 `CRC16` 值取模（`CRC16(key) % 16384`）确定所属槽位。
  - **哈希标签（Hash Tag）**：使用 `{}` 指定部分键名计算哈希值，强制相关键分配到同一槽。
  - **槽分配管理**：集群启动时，槽均匀分配到主节点，可通过 `CLUSTER ADDSLOTS` 手动分配或自动平衡。
- **节点角色**
  - **主节点（Master）**：负责处理槽的读写请求，参与故障选举。
  - **从节点（Replica）**：复制主节点数据，主节点故障时接替其槽。
  - **集群模式节点**：所有节点默认开启集群模式（`cluster-enabled yes`）。
- **集群拓扑**
  - **最小部署**：至少 3 个主节点（每个主节点至少 1 个从节点），共 6 个节点。
  - **节点发现**：节点通过 Gossip 协议交换状态信息（如 `MEET` 命令将节点加入集群）。

集群的工作机制

- **客户端请求路由**
  - **Smart Client**：客户端缓存槽与节点的映射关系，直接向目标节点发送请求，若节点返回`MOVED`重定向错误，更新缓存并重试。
  - **Dumb Client**：依赖代理（如 Redis Proxy）转发请求，客户端无需感知集群拓扑。

- **数据读写流程**
  - **键存在目标槽**：直接由负责该槽的节点处理。
  - **键不在目标槽**：返回 `MOVED` 错误，客户端重定向到正确节点。
  - **槽迁移中**：返回 `ASK` 错误，客户端临时重定向到迁移目标节点。
- **故障转移（Failover）**
  - **主观下线（PFAIL）**：节点 A 在 `cluster-node-timeout`（默认 15 秒）内未收到节点 B 的响应，标记 B 为 `PFAIL`。
  - **客观下线（FAIL）**：超过半数主节点确认节点 B 不可达，标记为 `FAIL`，触发故障转移。
  - **从节点选举**：从节点发起选举，获得多数主节点投票后成为新主节点；接管原主节点的槽，并广播更新集群配置。
- **数据迁移与平衡**
  - **手动迁移**：使用 `CLUSTER SETSLOT <slot> IMPORTING/MIGRATING` 命令迁移槽。
  - **自动平衡**：通过 `redis-cli --cluster rebalance` 自动调整槽分布，均衡负载。

集群的优缺点

- **优点**
  - **水平扩展**：支持 TB 级数据和高并发访问。
  - **高可用**：自动故障转移，数据多副本存储。
  - **去中心化**：无单点故障，节点自治。
- **缺点**
  - **功能限制**：不支持跨槽事务、部分命令受限（如 `KEYS *`）。
  - **运维复杂度**：需管理分片、迁移、节点扩缩容。
  - **客户端兼容性**：需使用集群感知的客户端或代理。

生产环境建议

- **部署与配置**
  - **节点规划**：主节点至少 3 个，跨物理机或可用区部署，从节点数 ≥ 主节点数。
  - **网络优化**：确保节点间低延迟通信，避免跨地域部署。
  - **内存管理**：监控节点内存使用，避免数据倾斜导致单个节点过载。
- **数据均衡**
  - **预分片**：设计键时使用哈希标签，确保相关数据集中。
  - **定期平衡**：使用 `redis-cli --cluster rebalance` 调整槽分布。
- **监控与故障排查**
  - **关键指标**：

    ```
    CLUSTER INFO          # 查看集群健康状态
    CLUSTER NODES         # 查看节点角色、槽分配、状态
    INFO memory           # 监控内存使用
    ```

  - **日志分析**：关注 `CLUSTERDOWN` 告警和节点超时事件。

常用集群命令

- **CLUSTER NODES**：查看集群节点信息。
- **CLUSTER INFO**：检查集群状态。
- **CLUSTER FAILOVER**：手动故障转移（从节点执行）。

Sentinel 与 Cluster 的对比

特性	Sentinel	Cluster
数据分片	不支持，需客户端或代理分片。	内置分片（16384 槽）。
高可用	主从复制 + 故障转移。	每个分片主从复制，自动故障转移。
扩展性	垂直扩展（主节点性能瓶颈）。	水平扩展（支持大规模数据集和高吞吐）。
适用场景	中小规模，非分片架构。	大规模数据，高并发场景。

Redis复制

主从复制的建立流程

- Redis 复制的建立分为 **全量同步（Full Sync）** 和 **部分同步（Partial Sync）** 两种模式，优先尝试部分同步以减少资源消耗。
- **从节点初始化连接**
  - **命令触发**：从节点执行 `SLAVEOF <master-ip> <master-port>`，开启复制流程。
  - **连接主节点**：从节点向主节点发起连接，发送 `PING` 确认网络可达性。
- **身份验证（可选）**：若主节点配置了 `requirepass`，从节点需发送 `AUTH <password>` 完成认证。
- **同步数据集**
  - **全量同步（SYNC）**：

    > 主节点生成当前数据的 **RDB 快照**，通过子进程写入磁盘。
    >
    > 将 RDB 文件发送给从节点，同时缓存期间的写命令至 **复制缓冲区（Replication Buffer）**。
    >
    > 从节点接收 RDB 并加载到内存，再执行缓冲区中的写命令，追上主节点状态。

  - **部分同步（PSYNC）**：

    > 从节点发送 `PSYNC <replid> <offset>`，携带自身记录的复制 ID 和偏移量。
    >
    > 主节点检查复制 ID 和偏移量是否匹配历史记录：
    >
    > **匹配**：发送 `+CONTINUE`，传输从偏移量之后的写命令（利用 **复制积压缓冲区**）。
    >
    > **不匹配**：触发全量同步（`+FULLRESYNC`）。

- **命令传播（Command Propagation）**：同步完成后，主节点持续将写命令发送给从节点，保持数据一致。

复制的核心机制

- **复制ID与偏移量**
  - **复制ID（Replication ID）**：主节点的唯一标识，每次主节点重启或角色变更时生成新 ID。
  - **偏移量（Offset）**：主从节点各自维护一个偏移量，记录已复制的数据量。

- **复制积压缓冲区（Replication Backlog）**
  - **作用**：主节点维护一个固定大小的环形缓冲区（默认 1MB），缓存最近的写命令。
  - **触发部分同步**：若从节点的偏移量在缓冲区范围内，直接发送增量数据。
  - **配置参数**

    ```
    repl-backlog-size 1mb     # 缓冲区大小
    repl-backlog-ttl 3600     # 主节点无连接时缓冲区保留时间（秒）
    ```

- **心跳检测与断线重连**
  - **心跳机制**：主从节点定期确认存活状态和复制进度；主节点超时未收到心跳（默认 60 秒）则认为从节点下线。
  - **断线处理**：从节点重连后尝试部分同步，失败则触发全量同步。

复制拓扑与高级特性

- **级联复制（主-从-从）**
  - **场景**：主节点连接过多从节点时，可通过级联复制分摊压力。
  - **配置**：将从节点（Slave A）作为另一从节点（Slave B）的主节点。
- **延迟副本（Lagging Replica）**
  - **作用**：人为设置从节点延迟同步，用于误操作恢复（需第三方工具支持）。
  - **实现**：通过 `slave-repl-delay` 配置延迟时间（Redis 自身不原生支持，需外部控制）。

复制的问题与优化

- **全量同步的资源消耗**
  - **问题**：大数据集时生成和传输 RDB 文件会阻塞主节点并占用带宽。
  - **优化**：增大 `repl-backlog-size` 减少全量同步概率；使用无盘同步（`repl-diskless-sync yes`），但需主节点内存充足。
- **复制延迟**
  - **原因**：网络延迟、从节点负载过高或主节点写入压力大。
  - **监控**：通过 `INFO replication` 的 `slave_repl_offset` 与 `master_repl_offset` 差值判断延迟。
  - **优化**：提升网络带宽，减少主从节点跨地域部署；限制主节点写入速率（如使用管道批量写入）。
- **数据不一致**
  - **原因**：主从网络中断导致部分数据未同步。
  - **检测**：使用 `redis-cli --slave` 模拟从节点检查数据差异。
  - **修复**：手动触发全量同步（`SLAVEOF NO ONE` + 重新配置复制）。

第四部分 高级功能与扩展

事务

事务的核心命令

- **MULTI**：标记事务开始，后续命令按顺序入队，暂不执行。
- **EXEC**：执行事务队列中的所有命令，返回各命令的结果。
- **DISCARD**：取消事务，清空命令队列。
- **WATCH <key>**：监视指定键，若事务执行前键被修改，则事务终止（类似 CAS 机制）。
- **UNWATCH**：取消所有 `WATCH` 监控。

事务的特性

- **原子性**：`EXEC` 执行时，所有命令按顺序一次性执行，不会被其他客户端命令打断。
- **无回滚**：若命令入队时语法错误，事务直接拒绝执行；运行时错误（如对字符串执行 `INCR`）会继续执行后续命令。
- **隔离性**：事务执行期间不会被其他客户端操作干扰（单线程模型保证）。
- **无持久性**：是否持久化取决于 Redis 的持久化配置（RDB/AOF）。

适用场景

- **批量操作原子性**：如批量更新计数器、配置项。
- **简单 CAS 操作**：结合 `WATCH` 实现乐观锁，避免竞态条件（如库存扣减、余额转账）。
- **非回滚型任务**：无需回滚的连续操作（如日志记录、状态标记）。

生产环境建议

- **避免长事务**：事务队列命令过多会阻塞其他客户端，影响性能。
- **优先使用 Lua**：复杂逻辑用 Lua 脚本替代事务，减少网络开销。
- **合理使用 WATCH**：监控最少数量的键，降低冲突概率。

Lua脚本

Lua 脚本的核心优势

- **原子性**：脚本整体执行，期间不会被其他命令中断，保证数据一致性。
- **减少网络开销**：合并多个操作到单个脚本，减少客户端与服务端的通信次数。
- **复杂逻辑支持**：支持条件判断、循环、变量等编程结构，灵活处理业务逻辑。
- **高性能**：脚本在服务端执行，避免多次网络往返，提升吞吐量。

Lua 脚本的执行方式

- **EVAL 命令格式**：`EVAL <script> <numkeys> [KEYS...] [ARGV...]`
  - **script**：Lua 脚本代码。
  - **numkeys**：键参数的数量（`KEYS` 数组长度）。
  - **KEYS**：键名列表（避免硬编码，通过参数传递）。
  - **ARGV**：额外参数列表（如数值、标志位）。
- **缓存脚本执行**
  - **预加载脚本**：`SCRIPT LOAD "return redis.call('GET', KEYS[1])"`

- **通过 SHA1 执行**：`EVALSHA abcdef12345... 1 user:1`

Lua 脚本与 Redis 命令的交互

- **调用 Redis 命令**
  - **redis.call()**：执行 Redis 命令，若命令出错（如键不存在），直接返回错误并终止脚本。

    ```
    local value = redis.call('GET', KEYS[1])
    ```

  - **redis.pcall()**：执行 Redis 命令，出错时返回 Lua 错误表，脚本继续执行。

    ```
    local ok, result = pcall(redis.call, 'GET', KEYS[1])
    ```

- **参数传递与返回值**
  - **键和参数分离**：`KEYS` 和 `ARGV` 是 Lua 表（数组），索引从 1 开始;严格区分键和参数，避免逻辑混乱。
  - **返回值处理**：脚本最终返回值会被转换为 Redis 协议格式（Lua表转为多行回复）；支持返回`nil`对应（Redis的 `NULL` 回复）。

Lua 脚本的使用原则

- **原子性与阻塞**
  - **长时间脚本阻塞**：脚本执行期间阻塞其他操作，需避免复杂循环或大量计算；单个脚本执行时间控制在ms级，必要时拆分逻辑。
- **沙盒环境限制**
  - **禁用危险操作**：无法访问文件系统、网络或调用外部命令；禁止定义全局变量（需使用 `local` 声明局部变量）。
- **脚本复用性**
  - **参数化设计**：键和参数通过 `KEYS` / `ARGV` 传递，避免硬编码，提高脚本通用性。
  - **版本兼容性**：不同 Redis 版本支持的 Lua 库函数可能不同（如 `redis.breakpoint()` 仅限调试模式）。

排序

排序参数详解

- **ASC/DESC**：升序（默认）或降序排序。
- **ALPHA**：按字典序排序字符串元素。
- **BY <pattern>**：按外部键的值排序（如 `user:*:age`）。
- **GET <pattern>**：排序后返回外部键的值（可多次使用）。
- **LIMIT offset count**：分页获取结果（类似 SQL 的 `LIMIT`）。
- **STORE <key>**： 将排序结果存入新键（列表类型）。

排序的工作原理

- **数据加载**：从列表、集合或有序集合中提取元素；若使用 `BY` 或 `GET`，加载外部键的值到内存。
- **排序执行**：默认按数值比较（`ALPHA` 启用字符串比较）；使用快速排序算法，时间复杂度 **O(N log N)**。
- **结果处理**：应用 `LIMIT` 分页；按 `GET` 返回指定数据或直接返回元素；若指定 `STORE`，将结果保存为新列表。

发布与订阅

核心机制

- **角色**：
  - **发布者（Publisher）**：通过 `PUBLISH <channel> <message>` 发送消息到频道。
  - **订阅者（Subscriber）**：通过 `SUBSCRIBE <channel>` 订阅频道，或 `PSUBSCRIBE <pattern>` 按模式匹配订阅（如 `news.*`）。
  - **退订**：`UNSUBSCRIBE` 或 `PUNSUBSCRIBE` 取消订阅。
- **消息传递**：消息从发布者直接推送到所有订阅者，无中间存储；消息**不可回溯**，订阅者断开后重连会丢失未连接期间的消息。

关键命令与行为

- **SUBSCRIBE channel1**：订阅指定频道。
- **PSUBSCRIBE news.***：按模式订阅所有以 `news.` 开头的频道。
- **PUBLISH channel1 "Hello"**：向 `channel1` 发布消息 "Hello"。
- **UNSUBSCRIBE**：退订所有频道。

慢查询日志

核心配置参数

- **slowlog-log-slower-than**
  - **作用**：定义慢查询的时间阈值（单位：微秒，1秒=1,000,000微秒）。
  - **示例**

    ```
    config set slowlog-log-slower-than 10000  # 记录超过10毫秒的命令
    ```

- **slowlog-max-len**
  - **作用**：设置慢查询日志的最大条目数（内存中存储，先进先出）。
  - **示例**

    ```
    config set slowlog-max-len 500  # 最多保存500条慢查询记录
    ```

查看与分析慢查询日志

- **SLOWLOG GET [n]**：查看最新的n条记录（默认全部）。
- **SLOWLOG LEN`**：查看当前日志条目数。
- **SLOWLOG RESET**：清空慢查询日志。

监视器

MONITOR 的功能与使用

- **启用监控**：客户端执行 `MONITOR` 后，Redis 会将该连接转为监控模式，实时打印所有命令及其参数：
- **输出格式**：
  - **时间戳**：命令执行的精确时间（秒.微秒）。
  - **客户端信息**：数据库编号、客户端 IP 和端口。
  - **命令内容**：完整的命令及参数。

MONITOR 的性能影响

- **高并发写入**： 每个命令都会触发 MONITOR 输出，占用大量 CPU 和带宽，可能拖慢 Redis 整体性能。
- **监控客户端网络延迟**：MONITOR 输出量大时，可能阻塞客户端连接，导致其他操作延迟。
- **内存消耗**：Redis 需为每个监控连接维护输出缓冲区，内存压力增大。

附录

Redis命令参考

数据类型相关命令

- **SET**：设置键值，支持过期时间、不存在/存在条件。
- **GET**：获取键值，若键不存在返回 `nil`。
- **INCR/DECR**：将键的整数值增1/减1，键不存在时初始化为0。
- **MSET/MGET**：批量设置/获取多个键值。
- **LPUSH/RPUSH**：从列表左/右侧插入元素。
- **LPOP/RPOP**：移除并返回列表左/右侧第一个元素。
- **LRANGE**：获取列表指定范围的元素。
- **HSET/HGET**：设置/获取哈希字段值。
- **HGETALL**：获取哈希所有字段和值。
- **HINCRBY**：哈希字段整数值增加指定数值。
- **SADD/SREM**：添加/移除集合元素。
- **SMEMBERS**：返回集合所有元素（慎用大集合）。
- **SINTER/SUNION**：返回多个集合的交集/并集。
- **ZADD**：添加带分值的成员到有序集合。
- **ZRANGE/ZREVRANGE**：按分值升序/降序返回有序集合成员。
- **ZINCRBY**：增加有序集合成员的分值。

事务与脚本

- **MULTI**：开启事务，后续命令入队。
- **EXEC**：执行事务队列中的所有命令。
- **WATCH/UNWATCH**：监视键变化，若被修改则事务中止。
- **EVAL**：执行Lua脚本，支持原子性操作。
- **EVALSHA**：通过SHA1执行缓存的脚本。

- **SCRIPT LOAD**：加载脚本到缓存，返回SHA1摘要。

持久化与备份

- **SAVE**：同步生成RDB快照，阻塞主线程。
- **BGSAVE**：异步生成RDB快照，子进程执行。
- **LASTSAVE**：返回最后一次成功生成RDB的时间戳。
- **BGREWRITEAOF**：异步重写AOF文件，优化体积。
- **APPEND**：追加值到键（底层AOF操作）。

集群与高可用

- **SLAVEOF/REPLICAOF**：将当前节点设置为指定主节点的从节点。
- **CLUSTER INFO**：返回集群基本信息（状态、槽分配）。
- **CLUSTER NODES**：列出集群所有节点的详细信息。
- **CLUSTER ADDSLOTS**：将指定哈希槽分配给当前节点。
- **SENTINEL MASTERS**：列出被监控的主节点信息。
- **SENTINEL GET-MASTER-ADDR-BY-NAME**：获取主节点地址。

系统管理与监控

- **CLIENT LIST**：列出所有客户端连接信息。
- **CLIENT KILL**：关闭指定客户端连接。
- **INFO**：返回服务器状态信息（内存、CPU、复制等）。
- **SLOWLOG**：查看慢查询日志。
- **MONITOR**：实时监控所有命令（生产环境慎用）。

高级功能

- **SUBSCRIBE/PUBLISH/UNSUBSCRIBE**：订阅/发布/取消订阅频道消息。
- **GEOADD**：添加地理位置坐标。
- **GEODIST**：计算两地距离。
- **GEORADIUS**：返回指定半径内的位置。

其他实用命令

- **KEYS/SCAN**：查找匹配模式的键（`SCAN` 非阻塞迭代）。
- **TTL/PTTL**：获取键的剩余过期时间（秒/毫秒）。
- **EXPIRE/PEXPIRE**：设置键的过期时间（秒/毫秒）。
- **TYPE**：返回键的数据类型。
- **DEL**：删除指定键。

配置参数详解

网络与连接

- **bind**
  - **作用**：指定 Redis 监听的网络接口（IP 地址）。
  - **默认值**：`bind 127.0.0.1`（仅本地访问）。
  - **生产建议**：绑定内网 IP（如 `bind 192.168.1.10`），避免暴露到公网。
- **protected-mode**
  - **作用**：是否启用保护模式（无密码且未绑定 IP 时拒绝远程连接）。
  - **默认值**：`yes`。
  - **生产建议**：若需远程访问且未设密码，设置为 `no`，但需配合防火墙策略。
- **port**
  - **作用**：服务监听端口。
  - **默认值**：`6379`。
  - **建议**：修改默认端口（如 `6380`）以降低扫描攻击风险。
- **tcp-keepalive**
  - **作用**：TCP 心跳检测间隔（秒），防止连接中断。
  - **默认值**：`300`（5 分钟）。
  - **建议**：设为 `60`（1 分钟）以更快检测断连。

内存管理

- **maxmemory**
  - **作用**：最大内存限制（如 `4GB`）。
  - **默认值**：无限制。
  - **生产建议**：设为物理内存的 70%-80%（如 `maxmemory 8gb`）。
- **maxmemory-policy**
  - **作用**：内存满时的数据淘汰策略。
  - **noeviction**（默认）：拒绝写入，返回错误。
  - **allkeys-lru**：淘汰最近最少使用的键。
  - **volatile-lfu**：淘汰访问频率最低的过期键。
  - **allkeys-random**：随机淘汰任意键。
  - **生产建议**：读多写少场景用 `allkeys-lru`，需精确控制用 `volatile-ttl`。
- **maxmemory-samples**
  - **作用**：LRU/LFU 淘汰算法采样精度（越大越准，但 CPU 消耗高）。
  - **默认值**：`5`。
  - **建议**：设为 `10` 平衡性能与精度。

RDB快照

- **save**
  - **作用**：触发 RDB 持久化的条件（格式：`save <seconds> <changes>`）。
  - **生产建议**：根据数据重要性调整（如 `save 300 100`）。
- **stop-writes-on-bgsave-error**
  - **作用**：RDB 持久化失败时是否拒绝写入。
  - **默认值**：`yes`。
  - **建议**：保持默认，防止数据不一致。

AOF 日志

- **appendonly**
  - **作用**：是否启用 AOF 持久化。
  - **默认值**：`no`。
  - **生产建议**：设为 `yes` 确保数据安全（结合 RDB）。
- **appendfsync**
  - **作用**：AOF 文件同步策略。
  - **everysec**（默认）：每秒异步刷盘，最多丢失1秒数据。
  - **always**：每次写操作同步刷盘（性能差，数据零丢失）。
  - **no**：依赖操作系统刷盘（约30秒）。
  - **建议**：保持 `everysec` 平衡性能与安全。
- **auto-aof-rewrite-percentage / auto-aof-rewrite-min-size**
  - **作用**：AOF 重写触发条件（文件增长比例与最小大小）。
  - **默认值**：`100%`（翻倍）和 `64MB`。
  - **建议**：设为 `100% 1gb` 减少重写频率。

主从复制

- **replicaof**
  - **作用**：指定主节点（Redis 5.0+ 语法）。
  - **示例**：`replicaof 10.0.1.100 6379`。
- **repl-backlog-size**
  - **作用**：复制积压缓冲区大小（支持部分同步）。
  - **默认值**：`1mb`。
  - **建议**：设为 `512mb`，适应高写入场景。
- **repl-diskless-sync**
  - **作用**：无盘同步（直接发送 RDB 到从节点，不落盘）。
  - **默认值**：`no`。
  - **适用场景**：主节点磁盘 I/O 瓶颈时启用，需充足内存。

Redis 集群

- **cluster-enabled**
  - **作用**：启用集群模式。
  - **默认值**：`no`。
  - **集群必开**：`cluster-enabled yes`。
- **cluster-node-timeout**
  - **作用**：节点通信超时（毫秒）。
  - **默认值**：`15000`。
  - **建议**：网络不稳定时设为 `20000`。

Sentinel（哨兵）

- **sentinel monitor**
  - **作用**：定义监控的主节点。
  - **示例**：`sentinel monitor mymaster 10.0.1.100 6379 2`（quorum=2）。
- **sentinel parallel-syncs**
  - **作用**：故障转移后同时同步的从节点数。
  - **默认值**：`1`。
  - **建议**：根据主节点负载调整（如 `2`）。